CODE4LIFE TECHNOLOGY

LƯU LẠI ĐỂ ÔN TẬP

CẨM NANG OOP

BẰNG TIẾNG VIỆT • 45 TRANG

```python
class BadStack(list):
    pass

s = BadStack([1, 2])
s.insert(0, 99)
print(s)              # [99, 1, 2]
```

Chạy đúng. Nhưng thiết kế sai. Vì sao?

Class | 4 tính chất | SOLID | Patterns | Phỏng vấn

Code4Life Technology - Tào Việt Đức

Bản đồ cẩm nang

Đọc theo thứ tự nếu mới học. Đã biết rồi thì nhảy tới phần cần ôn.

1 VÌ SAO CẦN OOP Trang 3

2 CLASS VÀ OBJECT Trang 5

3 4 TÍNH CHẤT Trang 12

4 QUAN HỆ GIỮA CÁC CLASS Trang 20

5 SOLID Trang 24

6 DESIGN PATTERNS Trang 30

7 THỰC CHIẾN Trang 36

8 ÔN TẬP Trang 40

Code mẫu dùng Python. Tư tưởng dùng được cho Java, C++, C#.

1 Vì sao cần OOP?

Khi dữ liệu và hàm nằm rời nhau, ai cũng có thể làm sai dữ liệu.

```python
def deposit(account, amount):
    account["balance"] += amount

acc = {"owner": "An", "balance": 100}
deposit(acc, 50)
acc["balance"] = -999        # không có gì ngăn lại
print(acc["balance"])
```

KẾT QUẢ KHI CHẠY

```
-999
```

- ✓ OOP gom dữ liệu và hành vi vào cùng một object.
- ✓ Quy tắc nằm cạnh dữ liệu nên khó bị bỏ qua.
- ✓ Thêm loại mới thường ít phải sửa code cũ hơn.

NÊN DÙNG KHI

Có nhiều đối tượng, trạng thái và quy tắc nghiệp vụ.

CHƯA CẦN KHI

Script ngắn, xử lý dữ liệu một chiều, hàm đơn giản đã đủ rõ.

4 tính chất trong 1 trang

Tính chất	Ý chính	Ví dụ chiếc xe
Đóng gói	Dữ liệu đi cùng quy tắc của nó	Động cơ nằm trong vỏ, không chỉnh trực tiếp
Trừu tượng	Nói việc cần làm, giấu cách làm	Vặn ga là chạy, không cần biết động cơ
Kế thừa	Dùng lại code theo quan hệ "là một"	Xe điện là một loại xe
Đa hình	Cùng lời gọi, khác cách xử lý	Cùng lệnh tăng tốc, xe xăng và xe điện chạy khác nhau

OOP không phải là viết thật nhiều class. Mục tiêu là code dễ hiểu, dễ sửa và dễ mở rộng.

2 Class và Object

Class mô tả một loại đối tượng. Object là một cá thể của nó.

```
class Student:
    def __init__(self, name):
        self.name = name          # thuộc tính
        self.scores = []

    def add_score(self, score):   # phương thức
        self.scores.append(score)

an = Student("An")
binh = Student("Bình")
an.add_score(9)
print(an.name, an.scores)
print(binh.name, binh.scores)
```

KẾT QUẢ KHI CHẠY

```
An [9]
Bình []
```

- ✓ Thuộc tính là dữ liệu. Phương thức là hành vi.
- ✓ Mỗi object có trạng thái riêng: điểm của An không ảnh hưởng Bình.

3 self và __init__

Hiểu `self` là hiểu cách một phương thức biết nó đang làm việc với object nào.

```
class Student:
    def __init__(self, name):
        print("Khởi tạo", name)
        self.name = name

    def hello(self):
        return f"Mình là {self.name}"

an = Student("An")
print(an.hello())
print(Student.hello(an))    # cách viết đầy đủ
```

KẾT QUẢ KHI CHẠY

```
Khởi tạo An
Mình là An
Mình là An
```

- `an.hello()` thực chất là `Student.hello(an)`.
- `__init__` chạy ngay sau khi object được tạo, để gán trạng thái ban đầu.
- Tên `self` chỉ là quy ước. Java và C++ dùng `this`.

NGUYÊN TẮC

Object vừa tạo xong phải ở trạng thái hợp lệ, không bắt người dùng gọi thêm hàm thiết lập.

4 Thuộc tính class và object

Đặt nhầm một dòng, mọi object dùng chung một danh sách.

```
class Bad:
    items = []                  # của class: dùng chung

class Good:
    def __init__(self):
        self.items = []         # của object: riêng

a, b = Bad(), Bad()
a.items.append(1)
print(b.items)

c, d = Good(), Good()
c.items.append(1)
print(d.items)
```

KẾT QUẢ KHI CHẠY

```
[1]
[]
```

	Thuộc tính class	Thuộc tính object
Khai báo	Ngay trong thân class	Trong `__init__` qua `self`
Phạm vi	Dùng chung cho mọi object	Mỗi object một bản
Nên dùng	Hằng số, cấu hình chung	Trạng thái riêng

5 3 loại phương thức

```python
class Date:
    def __init__(self, d, m, y):
        self.d, self.m, self.y = d, m, y

    def show(self):                    # instance method
        return f"{self.d}/{self.m}/{self.y}"

    @classmethod
    def parse(cls, text):              # class method
        return cls(*map(int, text.split("-")))

    @staticmethod
    def is_leap(y):                    # static method
        return y % 4 == 0 and (y % 100 != 0 or y % 400 == 0)

print(Date.parse("6-11-2026").show())
print(Date.is_leap(2024), Date.is_leap(1900))
```

KẾT QUẢ KHI CHẠY

```
6/11/2026
True False
```

Loại	Nhận vào	Dùng khi
Instance	`self`	Cần đọc hoặc đổi trạng thái object
Class	`cls`	Tạo object theo cách khác
Static	Không có	Hàm tiện ích liên quan tới class

6 Phương thức đặc biệt

Cho object của bạn dùng được `print`, `==` và `+` như kiểu có sẵn.

```
class Money:
    def __init__(self, amount):
        self.amount = amount

    def __repr__(self):                 # cho lập trình viên
        return f"Money({self.amount})"

    def __str__(self):                  # cho người đọc
        return f"{self.amount:,} đ"

    def __eq__(self, other):            # toán tử ==
        return self.amount == other.amount

    def __add__(self, other):           # toán tử +
        return Money(self.amount + other.amount)

a, b = Money(1000), Money(2500)
print(a + b)
print(repr(a))
print(a == Money(1000))
```

KẾT QUẢ KHI CHẠY

```
3,500 đ
Money(1000)
True
```

LƯU Ý

Viết `__eq__` mà không viết `__hash__` thì object không dùng được trong set hoặc làm key của dict.

7 Tham chiếu và sao chép

Gán biến không tạo object mới. Nó chỉ thêm một cái tên trỏ vào object cũ.

```
import copy

class Box:
    def __init__(self, items):
        self.items = items

a = Box([1])
b = a                       # cùng một object
b.items.append(2)
print(a.items, a is b)

c = copy.copy(a)            # bản sao nông
d = copy.deepcopy(a)        # bản sao sâu
c.items.append(3)
print(a.items, d.items)
```

KẾT QUẢ KHI CHẠY

```
[1, 2] True
[1, 2, 3] [1, 2]
```

Cách	Object ngoài	Object bên trong
b = a	Dùng chung	Dùng chung
copy	Tạo mới	Dùng chung
deepcopy	Tạo mới	Tạo mới

`==` so sánh giá trị. `is` kiểm tra có phải cùng một object không.

8 dataclass

Class chủ yếu để chứa dữ liệu thì không cần viết tay từng dòng.

```python
from dataclasses import dataclass, field

@dataclass
class Product:
    name: str
    price: int
    tags: list = field(default_factory=list)

p = Product("Bút", 5000)
print(p)
print(p == Product("Bút", 5000))
```

KẾT QUẢ KHI CHẠY

```
Product(name='Bút', price=5000, tags=[])
True
```

- ✓ Tự sinh `__init__`, `__repr__` và `__eq__`.
- ✓ `field(default_factory=list)` tránh lỗi list dùng chung.
- ✓ Thêm `frozen=True` để object không đổi được sau khi tạo.

NÊN DÙNG
Dữ liệu trả về từ API, cấu hình, bản ghi đơn giản.

TƯƠNG ĐƯƠNG
Java có `record`, Kotlin có `data class`.

9 Đóng gói

Giữ dữ liệu và quy tắc thay đổi dữ liệu ở cùng một chỗ.

```
class Account:
    def __init__(self, balance):
        self._balance = balance

    @property
    def balance(self):              # chỉ cho đọc
        return self._balance

    def withdraw(self, amount):
        if amount <= 0 or amount > self._balance:
            raise ValueError("Số tiền không hợp lệ")
        self._balance -= amount

acc = Account(100)
acc.withdraw(30)
print(acc.balance)
try:
    acc.withdraw(500)
except ValueError as e:
    print(e)
```

KẾT QUẢ KHI CHẠY

```
70
Số tiền không hợp lệ
```

- ✓ Code bên ngoài không tự trừ tiền, mà phải đi qua `withdraw()`.
- ✓ Quy tắc nằm ở một chỗ, nên chỉ cần sửa một lần.

HIỂU NHẦM HAY GẶP

Đóng gói không có nghĩa là viết getter, setter cho mọi thuộc tính.

10 Property và truy cập

```python
class Student:
    def __init__(self, age):
        self.age = age                  # đi qua setter

    @property
    def age(self):
        return self._age

    @age.setter
    def age(self, value):
        if not 0 < value < 120:
            raise ValueError("Tuổi không hợp lệ")
        self._age = value

s = Student(20)
try:
    s.age = -5
except ValueError as e:
    print(e, s.age)
```

KẾT QUẢ KHI CHẠY

```
Tuổi không hợp lệ 20
```

Mức	Python	Java, C++
Công khai	`name`	`public`
Nội bộ	`_name` (quy ước)	`protected`
Riêng tư	`__name` (đổi tên, vẫn truy cập được)	`private`

Python tin vào quy ước. Java và C++ do trình biên dịch kiểm tra.

11 Kế thừa

Class con dùng lại code của cha khi quan hệ thật sự là "là một".

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

    def pay(self):
        return self.salary

class Manager(Employee):
    def __init__(self, name, salary, bonus):
        super().__init__(name, salary)
        self.bonus = bonus

    def pay(self):                    # ghi đè
        return super().pay() + self.bonus

print(Employee("An", 10).pay())
print(Manager("Bình", 10, 5).pay())
```

KẾT QUẢ KHI CHẠY

```
10
15
```

- ✓ `super()` gọi phần việc của class cha, tránh chép lại code.
- ✓ Manager là một Employee, nên dùng được ở nơi cần Employee.

KIỂM TRA NHANH
Đọc thành câu "B là một A".
Nghe sai thì đừng kế thừa.

CẨN THẬN
Đổi class cha sẽ ảnh hưởng mọi class con.

12 super, MRO và đa kế thừa

Khi có nhiều class cha, Python tra phương thức theo một thứ tự cố định gọi là MRO.

```
class A:
    def hi(self):
        return "A"

class B(A):
    def hi(self):
        return "B > " + super().hi()

class C(A):
    def hi(self):
        return "C > " + super().hi()

class D(B, C):
    def hi(self):
        return "D > " + super().hi()

print(D().hi())
print([k.__name__ for k in D.__mro__])
```

KẾT QUẢ KHI CHẠY

```
D > B > C > A
['D', 'B', 'C', 'A', 'object']
```

Ngôn ngữ	Đa kế thừa class	Cách xử lý
Python	Có	Theo thứ tự MRO
C++	Có	Dùng `virtual` khi gặp hình kim cương
Java	Không	Implement nhiều interface

13 Đa hình

Cùng một lời gọi, mỗi object tự xử lý theo cách của nó.

```
class Circle:
    def __init__(self, r):
        self.r = r

    def area(self):
        return round(3.14 * self.r ** 2, 2)

class Square:
    def __init__(self, a):
        self.a = a

    def area(self):
        return self.a ** 2

for s in [Circle(2), Square(3)]:  # không cần if theo loại
    print(type(s).__name__, s.area())
```

KẾT QUẢ KHI CHẠY

```
Circle 12.56
Square 9
```

- Thêm hình mới chỉ cần thêm class, không phải sửa vòng lặp.
- Python không bắt buộc chung class cha, chỉ cần có `area()`. Đó là duck typing.
- Java và C++ thường cần chung class cha hoặc interface.

DẤU HIỆU NÊN DÙNG ĐA HÌNH

Chuỗi `if` theo loại đối tượng lặp lại ở nhiều nơi.

14 Override và Overload

```python
class Greeter:
    def hi(self, name):
        return "bản 1"

    def hi(self, name, times=1):   # đè bản 1
        return " ".join([f"Chào {name}"] * times)

g = Greeter()
print(g.hi("An"))
print(g.hi("An", 2))
print(len([k for k in vars(Greeter) if k == "hi"]))
```

KẾT QUẢ KHI CHẠY

```
Chào An
Chào An Chào An
1
```

	Ghi đè (override)	Nạp chồng (overload)
Là gì	Class con viết lại phương thức của cha	Cùng tên, khác tham số
Chọn lúc	Khi chạy	Khi biên dịch (Java, C++)
Python	Có	Không có, định nghĩa sau đè định nghĩa trước

TRONG PYTHON
Thay overload bằng tham số mặc định như `times=1`.

15 Trừu tượng

Bên gọi chỉ cần `pay()`, không cần biết trả bằng cách nào.

```
from abc import ABC, abstractmethod

class Payment(ABC):
    @abstractmethod
    def pay(self, amount): ...

class Card(Payment):
    def pay(self, amount):
        return f"Thẻ: {amount}"

class Wallet(Payment):
    def pay(self, amount):
        return f"Ví: {amount}"

for method in [Card(), Wallet()]:
    print(method.pay(100))
try:
    Payment()
except TypeError:
    print("Không tạo được object từ class trừu tượng")
```

KẾT QUẢ KHI CHẠY

```
Thẻ: 100
Ví: 100
Không tạo được object từ class trừu tượng
```

	Trừu tượng	Đóng gói
Trả lời	Object làm được gì?	Dữ liệu được bảo vệ thế nào?
Công cụ	Abstract class, interface	Quyền truy cập, property

16 Abstract hay Interface?

```python
from typing import Protocol

class Printable(Protocol):
    def to_text(self) -> str: ...

class Invoice:                      # không cần kế thừa
    def to_text(self):
        return "Hóa đơn 100"

def show(item: Printable):
    print(item.to_text())

show(Invoice())
```

KẾT QUẢ KHI CHẠY

```
Hóa đơn 100
```

	Abstract class	Interface (Java)	Protocol (Python)
Giữ trạng thái	Có	Không	Không cần
Code sẵn	Có	Có `default`	Không cần
Một class dùng	Một class cha (Java)	Nhiều interface	Nhiều protocol
Khai báo kế thừa	Bắt buộc	Bắt buộc	Không, chỉ cần khớp phương thức

CHỌN ABSTRACT CLASS
Khi các class con chia sẻ code và trạng thái.

CHỌN INTERFACE, PROTOCOL
Khi chỉ cần thống nhất "làm được gì".

17 5 kiểu quan hệ

Quan hệ	Nghĩa	Ví dụ
Kế thừa	Là một	Manager là một Employee
Phụ thuộc	Dùng tạm trong một hàm	Report nhận Printer làm tham số
Liên kết	Biết nhau, giữ tham chiếu	Giáo viên dạy học sinh
Tập hợp Aggregation	Có, nhưng phần sống độc lập	Lớp học có sinh viên
Kết hợp Composition	Có, phần sống cùng tổng thể	Đơn hàng có dòng đơn hàng

```
class Line:
    pass

class Order:                    # kết hợp: tự tạo phần của mình
    def __init__(self):
        self.lines = [Line()]

class Student:
    def __init__(self, name):
        self.name = name

class Classroom:                # tập hợp: nhận phần từ ngoài
    def __init__(self, students):
        self.students = students

an = Student("An")
room = Classroom([an])
del room
print(an.name)                  # An vẫn tồn tại
```

KẾT QUẢ KHI CHẠY

```
An
```

18 Đọc sơ đồ class UML

Account
- balance: int # owner: str
+ deposit(x): void + withdraw(x): void

- Ô trên là tên, giữa là thuộc tính, dưới là phương thức.
- `+` public, `-` private, `#` protected.
- Chữ nghiêng là class hoặc phương thức trừu tượng.



Kế thừa
Mũi tên rỗng trỏ về class cha



Hiện thực interface
Nét đứt, trỏ về interface



Liên kết
Class này giữ tham chiếu tới class kia



Phụ thuộc
Dùng tạm, đổi class kia có thể ảnh hưởng



Tập hợp
Thoi rỗng ở phía tổng thể, phần sống độc lập



Kết hợp
Thoi đặc ở phía tổng thể, phần sống cùng nó

19 Kế thừa hay Kết hợp?

Đáp án cho câu hỏi ở bìa: Stack kế thừa list nên nhận luôn cả `insert`.

```
class BadStack(list):                    # kế thừa sai
    pass

s = BadStack([1, 2])
s.insert(0, 99)                          # chen vào đáy stack
print(s)
```

KẾT QUẢ KHI CHẠY

```
[99, 1, 2]
```

```
class Stack:                             # kết hợp
    def __init__(self):
        self._items = []                 # Stack CÓ MỘT list

    def push(self, x):
        self._items.append(x)

    def pop(self):
        return self._items.pop()

s = Stack()
s.push(1)
s.push(2)
print(s.pop(), hasattr(s, "insert"))
```

KẾT QUẢ KHI CHẠY

```
2 False
```

Kế thừa: "là một"	Kết hợp: "có một"
Nhận hết phương thức của cha	Chỉ lộ ra thứ mình muốn

20 Dependency Injection

Nhận thứ mình cần từ bên ngoài, thay vì tự tạo bên trong.

```
class Mailer:
    def send(self, text):
        print("Gửi:", text)

class FakeMailer:                       # dùng khi test
    def __init__(self):
        self.sent = []

    def send(self, text):
        self.sent.append(text)

class Signup:
    def __init__(self, mailer):     # nhận từ ngoài
        self.mailer = mailer

    def register(self, email):
        self.mailer.send(f"Chào {email}")

Signup(Mailer()).register("an@mail.vn")
fake = FakeMailer()
Signup(fake).register("binh@mail.vn")
print(fake.sent)
```

KẾT QUẢ KHI CHẠY

```
Gửi: Chào an@mail.vn
['Chào binh@mail.vn']
```

- ✓ Test được `Signup` mà không gửi email thật.
- ✓ Đổi nhà cung cấp email mà không sửa `Signup`.

SOLID trong 1 trang

5 nguyên tắc giúp code dễ sửa hơn, kèm dấu hiệu vi phạm.

S

Single Responsibility

Một class chỉ nên có một lý do để thay đổi.

Dấu hiệu: class vừa tính tiền, vừa gửi email, vừa ghi file.

O

Open/Closed

Mở để mở rộng, đóng để sửa đổi.

Dấu hiệu: thêm một loại mới là phải sửa chuỗi if/elif cũ.

L

Liskov Substitution

Thay class con vào chỗ class cha, chương trình vẫn phải đúng.

Dấu hiệu: class con ném lỗi ở phương thức cha hứa có.

I

Interface Segregation

Đừng ép class phụ thuộc vào phương thức nó không dùng.

Dấu hiệu: interface quá to, nhiều phương thức để trống.

D

Dependency Inversion

Logic cấp cao phụ thuộc vào abstraction, không vào chi tiết.

Dấu hiệu: logic nghiệp vụ tự tạo thẳng một database cụ thể.

SOLID là định hướng, không phải luật. Hãy áp dụng khi code bắt đầu khó sửa.

S Single Responsibility

Một class chỉ nên có một lý do để thay đổi.

TRƯỚC: MỘT CLASS, BA LÝ DO ĐỂ SỬA

Class `Report` vừa tính tổng, vừa định dạng, vừa ghi file. Đổi mẫu báo cáo cũng có thể làm hỏng phần tính tiền.

```
class ReportCalculator:                 # lý do đổi: công thức
    def total(self, items):
        return sum(items)

class ReportFormatter:                  # lý do đổi: giao diện
    def render(self, total):
        return f"Tổng: {total:,} đ"

class ReportStore:                      # lý do đổi: nơi lưu
    def __init__(self):
        self.saved = []

    def save(self, text):
        self.saved.append(text)

total = ReportCalculator().total([1000, 2500])
store = ReportStore()
store.save(ReportFormatter().render(total))
print(store.saved)
```

KẾT QUẢ KHI CHẠY

```
['Tổng: 3,500 đ']
```

CÂU HỎI NÊN TỰ HỎI

Ai sẽ yêu cầu sửa class này? Nếu là nhiều bộ phận khác nhau, nên tách.

O Open/Closed

Thêm tính năng bằng cách thêm code mới, không sửa code đã chạy ổn.

TRƯỚC
Mỗi loại giảm giá mới lại thêm một nhánh `elif` vào hàm `checkout`.

```
class Discount:
    def apply(self, price):
        return price

class StudentDiscount(Discount):
    def apply(self, price):
        return price * 80 // 100

class VipDiscount(Discount):          # thêm mới, không sửa cũ
    def apply(self, price):
        return price * 70 // 100

def checkout(price, discount):
    return discount.apply(price)

print(checkout(100, Discount()))
print(checkout(100, StudentDiscount()))
print(checkout(100, VipDiscount()))
```

KẾT QUẢ KHI CHẠY

```
100
80
70
```

ĐỪNG LÀM QUÁ
Chỉ tách khi thật sự có nhiều biến thể. Chưa cần thì một `if` vẫn tốt hơn năm class.

L Liskov Substitution

Hình vuông là hình chữ nhật trong toán, nhưng chưa chắc đúng trong code.

```python
class Rectangle:
    def __init__(self, w, h):
        self.w, self.h = w, h

    def set_w(self, w):
        self.w = w

    def area(self):
        return self.w * self.h

class Square(Rectangle):
    def __init__(self, a):
        super().__init__(a, a)

    def set_w(self, w):              # đổi luôn cả chiều cao
        self.w = self.h = w

def stretch(r):                      # chỉ mong đổi chiều rộng
    r.set_w(10)
    return r.area()

print(stretch(Rectangle(5, 5)))
print(stretch(Square(5)))
```

KẾT QUẢ KHI CHẠY

```
50
100
```

- Cùng cạnh ban đầu là 5, nhưng class con cho kết quả khác điều cha hứa.
- Cách sửa: không để Square kế thừa Rectangle, cho cả hai cùng hiện thực một `Shape`.

I Interface Segregation

Nhiều interface nhỏ tốt hơn một interface to.

TRƯỚC

Interface `Machine` có cả in, quét và fax. Máy in thường buộc phải viết hàm quét rỗng.

```
from abc import ABC, abstractmethod

class Printer(ABC):
    @abstractmethod
    def print_doc(self): ...

class Scanner(ABC):
    @abstractmethod
    def scan(self): ...

class SimplePrinter(Printer):
    def print_doc(self):
        return "In"

class Copier(Printer, Scanner):
    def print_doc(self):
        return "In bản sao"

    def scan(self):
        return "Quét"

print(SimplePrinter().print_doc())
print(Copier().scan())
```

KẾT QUẢ KHI CHẠY

```
In
Quét
```

D Dependency Inversion

Nghiệp vụ không nên biết dữ liệu đang nằm ở MySQL hay ở đâu khác.

```python
from abc import ABC, abstractmethod

class UserRepo(ABC):                    # abstraction
    @abstractmethod
    def find(self, user_id): ...

class MemoryRepo(UserRepo):             # chi tiết, đổi được
    def __init__(self):
        self.data = {1: "An"}

    def find(self, user_id):
        return self.data.get(user_id, "Không có")

class UserService:                      # nghiệp vụ
    def __init__(self, repo: UserRepo):
        self.repo = repo

    def name(self, user_id):
        return self.repo.find(user_id)

service = UserService(MemoryRepo())
print(service.name(1), "|", service.name(2))
```

KẾT QUẢ KHI CHẠY

```
An | Không có
```

Dependency Inversion	Dependency Injection
Nguyên tắc: phụ thuộc vào abstraction	Kỹ thuật: truyền phụ thuộc từ ngoài vào

Bản đồ 23 pattern

Pattern là lời giải đã đặt tên cho bài toán thiết kế hay lặp lại, không phải thư viện để chép.

Khởi tạo (5)
Abstract Factory
Builder
Factory Method
Prototype
Singleton

Cấu trúc (7)
Adapter
Bridge
Composite
Decorator
Facade
Flyweight
Proxy

Hành vi (11)
Chain of Responsibility
Command,
Interpreter
Iterator, Mediator
Memento, Observer
State, Strategy
Template Method
Visitor

Bài toán	Pattern gợi ý
Tạo object mà không gắn chặt vào class cụ thể	Factory
Đổi thuật toán lúc chạy	Strategy
Một thay đổi cần báo cho nhiều nơi	Observer
Thêm tính năng mà không sửa class gốc	Decorator
Ghép hai giao diện không khớp nhau	Adapter

Bắt đầu từ vấn đề, rồi mới tới pattern. Đừng cố nhét pattern vào code.

21 Factory

Gom việc chọn class cần tạo vào một chỗ.

```python
class Email:
    def send(self):
        return "Gửi email"

class Sms:
    def send(self):
        return "Gửi SMS"

def make_sender(kind):
    senders = {"email": Email, "sms": Sms}
    return senders[kind]()

print(make_sender("sms").send())
print(make_sender("email").send())
```

KẾT QUẢ KHI CHẠY

```
Gửi SMS
Gửi email
```

- ✓ Code gọi không cần biết tên class cụ thể.
- ✓ Thêm kênh mới chỉ sửa ở một nơi.

LƯU Ý TÊN GỌI

Ví dụ này là Simple Factory. Factory Method của GoF để class con quyết định tạo gì.

DÙNG THẬT

Chọn cổng thanh toán, loại thông báo, trình đọc file theo định dạng.

22 Singleton

Bảo đảm chỉ có một object của class trong toàn chương trình.

```
class Config:
    _instance = None

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance

a, b = Config(), Config()
a.debug = True
print(a is b, b.debug)
```

KẾT QUẢ KHI CHẠY

```
True  True
```

HỢP VỚI

Cấu hình, log, kết nối dùng chung ở mức ứng dụng.

RỦI RO

Là trạng thái toàn cục: khó test, giấu phụ thuộc, dễ lỗi khi chạy đa luồng.

- ✓ Trong Python, một module vốn chỉ nạp một lần nên thường thay được Singleton.
- ✓ Nếu chỉ cần dùng chung, hãy tạo một object rồi truyền vào nơi cần.

Pattern này rất dễ bị lạm dụng. Hãy cân nhắc kỹ trước khi dùng.

23 Strategy

Đổi cách xử lý lúc chạy mà không sửa code gọi.

```
def standard(km):
    return 15_000

def express(km):
    return 15_000 + km * 2_000

class Cart:
    def __init__(self, shipping):
        self.shipping = shipping   # chiến lược được truyền vào

    def fee(self, km):
        return self.shipping(km)

print(Cart(standard).fee(5))
print(Cart(express).fee(5))
```

KẾT QUẢ KHI CHẠY

```
15000
25000
```

- ✓ `Cart` không chứa `if` theo loại giao hàng.
- ✓ Python truyền thẳng hàm được, không nhất thiết phải tạo class.
- ✓ Java và C++ thường dùng interface hoặc lambda.

DÙNG THẬT

Tính phí vận chuyển, chọn thuật toán sắp xếp, nén file, xác thực đăng nhập.

24 Observer

Một sự kiện xảy ra, mọi nơi đã đăng ký đều được báo.

```
class Channel:
    def __init__(self):
        self.subscribers = []

    def subscribe(self, callback):
        self.subscribers.append(callback)

    def post(self, title):
        for notify in self.subscribers:
            notify(title)

channel = Channel()
channel.subscribe(lambda t: print("An nhận:", t))
channel.subscribe(lambda t: print("Bình nhận:", t))
channel.post("Cẩm nang OOP")
```

KẾT QUẢ KHI CHẠY

```
An nhận: Cẩm nang OOP
Bình nhận: Cẩm nang OOP
```

- ✓ `Channel` không cần biết ai đang lắng nghe.
- ✓ Thêm người nhận mới không phải sửa `Channel`.

DÙNG THẬT

Sự kiện giao diện, thông báo, event bus, cập nhật giá.

CẨN THẬN

Quên hủy đăng ký dễ gây rò bộ nhớ, và luồng chạy khó theo dõi hơn.

25 Decorator và Adapter

```python
class Coffee:
    def cost(self):
        return 30

class Milk:                          # bọc thêm tính năng
    def __init__(self, drink):
        self.drink = drink

    def cost(self):
        return self.drink.cost() + 10

print(Milk(Milk(Coffee())).cost())
```

KẾT QUẢ KHI CHẠY

```
50
```

```python
class OldGateway:                    # không sửa được
    def make_payment(self, cents):
        return f"Đã trả {cents} xu"

class PayAdapter:                    # đổi cho khớp giao diện
    def __init__(self, gateway):
        self.gateway = gateway

    def pay(self, amount):
        return self.gateway.make_payment(amount * 100)

print(PayAdapter(OldGateway()).pay(5))
```

KẾT QUẢ KHI CHẠY

```
Đã trả 500 xu
```

DECORATOR
Giữ nguyên giao diện, thêm hành vi.

ADAPTER
Giữ nguyên hành vi, đổi giao diện.

26 6 lỗi thiết kế hay gặp

Lỗi	Dấu hiệu	Cách sửa
God Class	Một class làm đủ mọi việc	Tách theo trách nhiệm
Kế thừa sâu	Nhiều tầng class, sửa cha vỡ con	Chuyển sang kết hợp
Anemic Model	Class chỉ có getter, setter, logic nằm chỗ khác	Đưa logic về cạnh dữ liệu
Feature Envy	Phương thức dùng dữ liệu class khác nhiều hơn của mình	Chuyển sang class đó
Primitive Obsession	Dùng str, int cho tiền, email, số điện thoại	Tạo class nhỏ như Money, Email
If theo loại	Cùng một chuỗi if lặp ở nhiều nơi	Đa hình hoặc Strategy

Đây là tín hiệu nên xem lại, không phải luật. Hãy sửa khi nó thật sự làm bạn chậm đi.

27 Dự án mini: Đơn hàng

```python
from dataclasses import dataclass, field

@dataclass
class Item:
    name: str
    price: int
    qty: int = 1

    def total(self):
        return self.price * self.qty

class Percent:
    def __init__(self, rate):
        self.rate = rate

    def apply(self, amount):
        return amount * (100 - self.rate) // 100

@dataclass
class Order:
    discount: Percent
    items: list = field(default_factory=list)

    def add(self, item):
        self.items.append(item)

    def total(self):
        raw = sum(i.total() for i in self.items)
        return self.discount.apply(raw)

order = Order(Percent(10))
order.add(Item("Sách", 100_000, 2))
order.add(Item("Bút", 5_000))
print(order.total())
```

KẾT QUẢ KHI CHẠY

```
184500
```

28 Mở rộng dự án

Thêm giảm giá theo số tiền mà không sửa logic đơn hàng.

```
class Fixed:                            # loại giảm giá mới
    def __init__(self, value):
        self.value = value

    def apply(self, amount):
        return max(amount - self.value, 0)

class Order:                            # bản rút gọn
    def __init__(self, amount, discount):
        self.amount = amount
        self.discount = discount

    def total(self):
        return self.discount.apply(self.amount)

print(Order(205_000, Fixed(5_000)).total())
print(Order(3_000, Fixed(5_000)).total())
```

KẾT QUẢ KHI CHẠY

```
200000
0
```

Khái niệm	Nằm ở đâu trong dự án
Đóng gói	`Item` tự tính tổng của mình
Kết hợp	`Order` có danh sách `Item` và một giảm giá
Đa hình	Mọi loại giảm giá đều có `apply()`
Open/Closed	Thêm `Fixed` mà không sửa `total()`

29 Viết test cho class

Thiết kế tốt thì test dễ viết. Khó test thường là dấu hiệu nên xem lại thiết kế.

```
import unittest

class Cart:
    def __init__(self):
        self.prices = []

    def add(self, price):
        if price <= 0:
            raise ValueError("Giá không hợp lệ")
        self.prices.append(price)

    def total(self):
        return sum(self.prices)

class CartTest(unittest.TestCase):
    def test_total(self):
        cart = Cart()
        cart.add(10)
        cart.add(20)
        self.assertEqual(cart.total(), 30)

    def test_bad_price(self):
        with self.assertRaises(ValueError):
            Cart().add(0)

tests = unittest.defaultTestLoader.loadTestsFromTestCase(CartTest)
result = unittest.TestResult()
tests.run(result)
print(result.testsRun, "test,", len(result.failures), "lỗi")
```

KẾT QUẢ KHI CHẠY

```
2 test, 0 lỗi
```

Tóm tắt 1 trang

Từ khóa	Nhớ một câu
Class, Object	Bản mô tả và cá thể được tạo ra
Đóng gói	Dữ liệu đi cùng quy tắc của nó
Kế thừa	Chỉ dùng khi thật sự "là một"
Đa hình	Cùng lời gọi, khác cách xử lý
Trừu tượng	Nói việc cần làm, giấu cách làm
Kết hợp	Ưu tiên khi chỉ muốn dùng lại code
Dependency Injection	Nhận phụ thuộc từ ngoài vào
S	Một lý do để thay đổi
O	Thêm mới thay vì sửa cũ
L	Class con thay được class cha
I	Interface nhỏ và đúng việc
D	Phụ thuộc vào abstraction
Strategy	Đổi thuật toán lúc chạy
Observer	Một sự kiện, nhiều nơi nhận

? Đố bạn: in ra gì?

Tự đoán rồi comment đáp án của bạn, xong mới lật trang sau nhé.

Câu 1

```
class A:
    n = 0

a, b = A(), A()
a.n = 5
print(A.n, a.n, b.n)
```

Câu 2

```
class A:
    def who(self):
        return "A"
    def hi(self):
        return self.who()

class B(A):
    def who(self):
        return "B"

print(B().hi())
```

Câu 3

```
def add(x, box=[]):
    box.append(x)
    return box

add(1)
print(add(2))
```

Câu 4

```
class P:
    def __init__(self, x):
        self.x = x

print(P(1) == P(1))
print(P(1).x == P(1).x)
```

Đáp án

1

`0 5 0`

Lệnh `a.n = 5` tạo thuộc tính riêng cho `a`. `A.n` và `b.n` vẫn đọc giá trị của class.

2

`B`

`self` đang là object của `B`, nên `self.who()` gọi bản của `B`. Đó là đa hình.

3

`[1, 2]`

Giá trị mặc định chỉ tạo một lần khi định nghĩa hàm. Hãy dùng `box=None` rồi tạo list bên trong.

4

`False`, `True`

Chưa viết `__eq__` thì `==` so sánh hai object có phải là một không. Dòng dưới so sánh hai số.

Bạn đúng mấy câu? Comment số câu đúng nhé!

20 câu phỏng vấn (1/2)

1. OOP là gì?
Cách tổ chức chương trình quanh các object, mỗi object gồm dữ liệu và hành vi.

2. Class khác object thế nào?
Class là bản mô tả. Object là cá thể tạo từ class, có trạng thái riêng.

3. 4 tính chất của OOP là gì?
Đóng gói, kế thừa, đa hình và trừu tượng.

4. Trừu tượng khác đóng gói thế nào?
Trừu tượng giấu cách làm sau một giao diện. Đóng gói giữ dữ liệu đi cùng quy tắc.

5. Override khác overload thế nào?
Override: class con viết lại phương thức của cha. Overload: cùng tên, khác tham số.

6. Abstract class khác interface thế nào? (Java)
Abstract class giữ được trạng thái. Một class chỉ kế thừa một class, nhưng implement được nhiều interface.

7. Constructor dùng để làm gì?
Đưa object về trạng thái hợp lệ ngay khi tạo.

8. Static method khác instance method thế nào?
Instance method làm việc trên một object. Static method không cần object.

9. Đa hình lúc chạy hoạt động ra sao?
Phương thức được chọn theo kiểu thật của object khi chạy.

10. Vì sao nên ưu tiên kết hợp hơn kế thừa?
Ít ràng buộc hơn, dễ thay thành phần, không lộ hết phương thức của cha.

20 câu phỏng vấn (2/2)

11. Liskov Substitution nói gì?
Thay class con vào chỗ class cha thì chương trình vẫn phải đúng.

12. Dependency Injection là gì?
Truyền phụ thuộc từ ngoài vào thay vì tự tạo, giúp dễ thay và dễ test.

13. Shallow copy khác deep copy thế nào?
Shallow chép object ngoài, vẫn chung object con. Deep chép cả object con.

14. Vấn đề kim cương là gì?
Class kế thừa hai class cùng gốc nên mơ hồ về phương thức. Python giải bằng MRO.

15. Composition khác Aggregation thế nào?
Composition: phần sống cùng tổng thể. Aggregation: phần tồn tại độc lập.

16. Singleton có nhược điểm gì?
Tạo trạng thái toàn cục, khó test và dễ giấu phụ thuộc.

17. Khi nào dùng Strategy?
Khi cần đổi thuật toán lúc chạy mà không sửa code gọi.

18. God Class là gì?
Class ôm quá nhiều việc. Cách sửa là tách theo trách nhiệm.

19. `__str__` khác `__repr__` thế nào? (Python)
`__str__` cho người đọc. `__repr__` cho lập trình viên khi debug.

20. Khi nào không nên dùng OOP?
Script ngắn hoặc luồng xử lý dữ liệu đơn giản, nơi hàm đã đủ rõ.

Luyện tập theo cấp độ

1 **Beginner**
Viết `Rectangle` và `BankAccount` có kiểm tra dữ liệu đầu vào.

2 **Student**
Quản lý thư viện với `Book`, `Member`, `Loan`, vẽ sơ đồ UML trước khi code.

3 **Junior**
Đơn hàng có nhiều loại giảm giá, tách theo SOLID và viết unit test.

4 **Competitive**
Thiết kế ví điện tử dùng Strategy, Observer và Repository, giải thích được vì sao chọn.

LUYỆN TIẾP VỚI 4 REPO OOP CỦA CODE4LIFE
Beginner → Student → Junior → Competitive
`github.com/taovietducofficial`

Thấy chỗ nào chưa đúng? Anh em comment giúp để Code4Life sửa ở bản sau nhé.

Biên soạn: Tào Việt Đức, Code4Life Technology, với sự hỗ trợ của AI. Code chạy thử trên Python 3.14.